Entrega de Lab 2

Lab 5 - Pantalla TFT

L'objectiu principal d'aquest laboratori va ser aprendre a treballar amb la pantalla TFT i construir una base de treball més còmoda i visual per a les pràctiques següents. En una primera fase, es va muntar el circuit entre la pantalla i la Raspbery i se'n va provar el funcionament. En una segona fase, es va desenvolupar una llibreria pròpia per poder dibuixar per pantalla sense haver de treballar directament amb el protocol de baix nivell ni amb els registres específics del panell.

Circuit

Les connexions necessàries per a que la pantalla funcionés correctament van ser les següents:

- **Selecció del mode SPI**: **IM3-IM0** havíen de prendre el valor `0b0100` per indicar a la TFT que la comunicació es faria per SPI.

- **Alimentació**: connexions de **Vcc** i **GND** per al funcionament de la pantalla, i també **LedA** i **LedK** per al *backlighting*.

- **SPI**: connexions de **SCLK**, **MISO**, **MOSI** y **CS** (connectades als pins **GPIO 23**, **21**, **19** i **8** respectivament).

- **Control**: connexió del pin **RST** per reinicialitzar la TFT quan calgués.

Codi

Per comprovar que les connexions estiguessin ben fetes, es van executar tant el driver en Python com el de C, i es va verificar que la pantalla responia i que els píxels prenien valors aparentment aleatoris. Un cop confirmat que la TFT funcionava, es va desenvolupar una llibreria pròpia per amb funcions que permeten operacions més complexes que el *driver*. Entre altres coses, permet:

- Pintar tota la pantalla d'un color
- Dibuixar formes geomètriques
- Escriure texts
- Mostrar imatges

El vídeo "Video TFT.mp4" mostra com es renta la pantalla per complet i s'imprimeixen un text, un rectangle i una imatge.

Una part important del treball va ser l'optimització del dibuix de regions uniformes. En comptes d'escriure píxel per píxel repetint la configuració de coordenades a cada mostra, es defineix una regió i s'envien consecutivament els píxels de tota la zona amb el mateix color. La funció `SPI_TFT_region(x1, y1, x2, y2, color)` segueix aquesta idea: primer fixa els límits de la regió i després escriu el color repetidament sobre tota l'àrea. Això redueix notablement l'overhead de comunicació i és especialment útil per pintar rectangles, fons i altres superfícies homogènies.

Pel que fa al text, vam fer que es pogués imprimir amb diferents mides. En lloc de dibuixar sempre les lletres igual, vam afegir un paràmetre que permet escalar-les. Així, podem fer el mateix text més gran o més petit segons ens convingui, per exemple per destacar títols o fer etiquetes més petites, sense haver de definir una font nova cada vegada.

A nivell d'implementació, el que fem és dibuixar cada lletra a partir d'una plantilla bàsica (com una matriu de píxels) i, en lloc d'encendre un sol píxel per cada posició, en pintem un bloc més gran en funció de la mida escollida. És a dir, cada píxel de la lletra original es converteix en un petit quadrat de mida variable. Això és bastant senzill de fer i ens permet reutilitzar el mateix codi per a totes les mides sense complicar gaire la lògica.

Per a la part d'imatges, vam utilitzar la llibreria `stb_image.h`. És una bona tria perquè és molt lleugera i fàcil d'integrar (és un sol fitxer), i a més ens permet carregar formats com .png directament sense haver de fer conversions prèvies. A més utilitza intrínsecs SIMD, cosa que fa que processi les imatges a imprimir ràpidament. Tot això sumat simplifica bastant el codi i fa que treballar amb imatges sigui ràpid i còmode.

Temps d'execució

La següent comparació recull el temps de rentat complet de la pantalla amb un sol color en tres situacions diferents: el driver de Python, el driver de C i el driver de C amb l'optimització per regions.

Versió	**Temps d'execució**	**SpeedUp (respecte la versió anterior)**
Python	157.114667 s	-
C	3.059274 s	x51.36
C optimitzat	0.403267 s	x7.59

Lab 6 - Connexió PIC-Raspberry

L'objectiu d'aquest laboratori va ser implementar la comunicació entre el PIC i la Raspberry Pi mitjançant el protocol UART, combinant-la amb l'ús de la TFT del laboratori anterior. El resultat final havia de permetre que el PIC enviés informació a la Raspberry i que aquesta actualitzés la pantalla segons l'estat rebut.

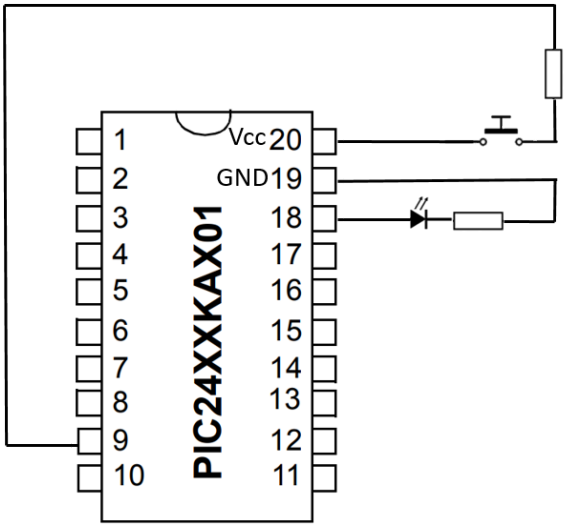
Circuit

Per facilitar la circuiteria, es van muntar dos circuits per separat abans d'unir-los:

- Circuit per al PIC, amb un botó i un LED
- Circuit per a la Raspberry Pi, connectada a la TFT

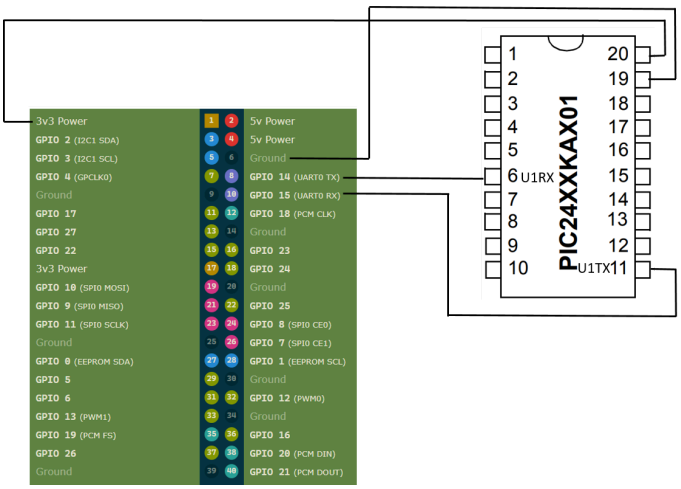
Quan tots dos muntatges funcionaven correctament per separat, es van connectar mitjançant UART per completar la interacció entre dispositius.

El circuit de la Raspberry és el mateix que al Lab 5. El del PIC és el següent:



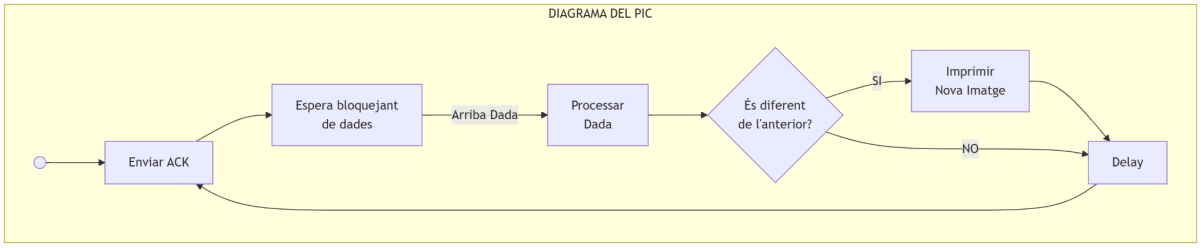
El LED del PIC tenia una funció purament indicadora: s'activava per *software* i servia per mostrar visualment si el botó estava actiu o no.

Les connexions entre Raspberry Pi i PIC van ser les següents:



Codi

El codi de la Raspberry segueix el següent diagrama de flux:



El codi del PIC, per altra banda, té una variable global que emmagatzema l'estat (actiu o apagat) del botó i canvia quan es detecta un flanc ascendent (ho hem fet tenint en compte el *bouncing*). Cada cop que arriba l'ACK de la Raspberry, salta una interrupció que envia l'estat del botó en aquell moment.

La comunicació UART per part del PIC es va implementar amb el hardware específic del microcontrolador, configurant els registres necessaris al principi de l'execució. Per la banda de la Raspberry, la comunicació es va fer amb crides POSIX, llegint els descriptors o fitxers de sistema que corresponen als buffers de la UART. D'aquesta manera, el programa pot gestionar la recepció i l'enviament de dades sèrie des d'un entorn Linux de manera estàndard.

La integració final va permetre que el PIC i la Raspberry treballessin de manera coordinada: el PIC detecta l'estat del botó i la Raspberry rep aquesta informació per actualitzar la interfície de la TFT. El sistema combina, per tant, entrada digital, comunicació sèrie i sortida gràfica en un únic muntatge funcional.

El vídeo "Video Comunicació UART.mp4" mostra el sistema, segons si s'activa o desactiva el botó, s'imprimeix una imatge de ON o de OFF a la pantalla.

Conclusions

Els dos laboratoris ens han servit per aprendre a treballar amb una pantalla TFT i amb el protocol UART per comunicar diferents microcontroladors. També ens ha mostrat la importància de com implementem el codi en aquests protocols, ja que una mala gestió pot provocar colls d'ampolla quan la velocitat és crítica.

Això ens dona una primera idea del que es pot aconseguir quan es combinen diversos microcontroladors amb tasques repartides, permetent construir un sistema més complex que seria difícil d'assolir amb un de sol.